
1. En orientación a objetos, ¿qué es la herencia y su relación con "A es-un B"? Explica las dos implicaciones principales: (1) compatibilidad de tipos y (2) herencia de estado y comportamiento. Pon un ejemplo en Java muy sencillo, donde un **Soldado** tiene un **nombre** (privado) y un método **saludar()** que muestra su nombre. Hay dos subtipos: un **Artillero**, que es capaz de disparar cohetes y un **Zapador** que pone minas, ambos heredan el atributo **nombre** y la capacidad de saludar. Además, y de forma específica, el artillero tiene un número de cohetes y el zapador un número de minas, accesibles mediante "getters" específicos. Respecto a la compatibilidad de tipos, aprovechémosla: crea un array de **Soldado**, mete varios de distinto tipo (son todos compatibles con **Soldado**). Recórrela y que todos te saluden.

En **orientación a objetos**, la **herencia** es el mecanismo por el cual una clase (subclase) **extiende** a otra (superclase), reutilizando su **estado (atributos)** y su **comportamiento (métodos)**, y pudiendo además **especializarse** con nuevos atributos o métodos.

Esta relación suele explicarse con la idea de "**A es-un B**" (*is-a*):

Si **Artillero** es un **Soldado**, entonces **Artillero** puede usarse allí donde se espere un **Soldado**.

Implicación 1: Compatibilidad de tipos (polimorfismo)

Gracias a la herencia:

- **Una instancia de una subclase es compatible con el tipo de la superclase.**
- Podemos tratar objetos distintos (artillero, zapador...) como si todos fueran **Soldado**.

Esto permite, por ejemplo:

- Guardarlos todos en un **Soldado[]**
- Llamar a métodos comunes (**saludar()**) sin saber el tipo concreto

Esto se conoce como **polimorfismo por subtipado** y cumple el principio de sustitución:

Un objeto hijo puede sustituir a su padre sin romper el programa.

Implicación 2: Herencia de estado y comportamiento

La subclase:

- **Hereda el estado** → atributos definidos en la superclase
- **Hereda el comportamiento** → métodos definidos en la superclase

En nuestro ejemplo:

- **nombre** y **saludar()** se definen **una sola vez** en **Soldado**
 - **Artillero** y **Zapador** los reutilizan tal cual
 - Cada subtipo añade **su propia información específica**
-

Ejemplo sencillo en Java

Clase base: **Soldado**

```
public class Soldado {  
  
    private String nombre;  
  
    public Soldado(String nombre) {  
        this.nombre = nombre;  
    }  
  
    public void saludar() {  
        System.out.println("Hola, soy el soldado " + nombre);  
    }  
}
```

- **nombre** es **privado** (encapsulación)
 - Todos los soldados saben **saludar**
-

Subtipo **Artillero**

```
public class Artillero extends Soldado {  
  
    private int cohetes;  
  
    public Artillero(String nombre, int cohetes) {  
        super(nombre);  
        this.cohetes = cohetes;  
    }  
  
    public int getCohetes() {  
        return cohetes;  
    }  
  
    public void dispararCohete() {  
        System.out.println("¡Disparando un cohete!");  
    }  
}
```

Subtipo **Zapador**

```
public class Zapador extends Soldado {  
  
    private int minas;
```

```
public Zapador(String nombre, int minas) {
    super(nombre);
    this.minas = minas;
}

public int getMinas() {
    return minas;
}

public void ponerMina() {
    System.out.println("¡Mina colocada!");
}
}
```

Aprovechando la compatibilidad de tipos

```
public class Ejercito {

    public static void main(String[] args) {

        Soldado[] soldados = new Soldado[3];

        soldados[0] = new Soldado("Carlos");
        soldados[1] = new Artillero("Lucía", 5);
        soldados[2] = new Zapador("Miguel", 3);

        for (Soldado s : soldados) {
            s.saludar();
        }
    }
}
```

¿Qué ocurre aquí?

- Todos los elementos del array son **compatibles con Soldado**
- No importa si son **Artillero** o **Zapador**
- El método **saludar()** funciona para **todos**

Resumen rápido

- **Herencia** modela relaciones “es-un”
 - Implica:
 1. ☒ **Compatibilidad de tipos** → polimorfismo
 2. ☒ **Reutilización de estado y comportamiento**
 - Permite escribir código **más reutilizable, flexible y claro**
-

2. Al crear los soldados concretos, ¿cuántos constructores se ejecutan y en qué orden? ¿Qué significa **super** dentro de un constructor? Si la clase base no tiene visible el constructor sin parámetros, ¿debo llamar a **super** siempre?

¿Cuántos constructores se ejecutan y en qué orden?

Siempre se ejecutan TODOS los constructores de la cadena de herencia, empezando por el más alto (la superclase) y terminando en la clase concreta.

En nuestro ejemplo:

```
Artillero a = new Artillero("Lucía", 5);
```

Constructores que se ejecutan

1. Constructor de **Soldado**
2. Constructor de **Artillero**

☒ **Dos constructores**, aunque tú solo llames explícitamente al de **Artillero**.

Orden de ejecución

☞ **De la superclase a la subclase:**

```
Soldado → Artillero
```

Esto es obligatorio porque:

- El objeto **primero tiene que ser un Soldado válido**
- Después puede convertirse en un Artillero

¿Qué significa **super** dentro de un constructor?

super(...) significa:

“Llama al **constructor de la clase padre**”

Ejemplo en **Artillero**:

```
public Artillero(String nombre, int cohetes) {  
    super(nombre);    // constructor de Soldado  
    this.cohetes = cohetes;  
}
```

Aquí ocurre lo siguiente:

1. `super(nombre)` inicializa la parte `Soldado` del objeto
2. Luego se inicializa la parte específica `Artillero`

Reglas importantes de `super(...)`

- ☒ **Debe ser la primera instrucción del constructor**
- ☒ Sirve para pasar parámetros al constructor padre
- ☒ Garantiza que el estado heredado se inicialice correctamente

¿Y si no escribo `super`?

Java **intenta hacerlo por ti**.

Si escribes:

```
public Artillero(String nombre, int cohetes) {  
    this.cohetes = cohetes;  
}
```

Java insertará automáticamente:

```
super(); // implícito
```

⚠ **PERO solo funciona si la clase padre tiene un constructor sin parámetros visible.**

¿Qué pasa si la clase base NO tiene constructor sin parámetros?

Por ejemplo, nuestra clase `Soldado`:

```
public class Soldado {  
  
    private String nombre;  
  
    public Soldado(String nombre) {  
        this.nombre = nombre;  
    }  
}
```

✗ **No existe `Soldado()`**

En este caso:

👉 **DEBES llamar a `super(...)` explícitamente**, o el código **no compila**.

Java no sabe cómo construir la parte **Soldado** del objeto.

Ejemplo de error típico

```
public Artillero(String nombre, int cohetes) {  
    // super(); ✗ no existe  
    this.cohetes = cohetes;  
}
```

✗ Error de compilación:

constructor Soldado() is undefined

☑ Solución correcta:

```
public Artillero(String nombre, int cohetes) {  
    super(nombre);  
    this.cohetes = cohetes;  
}
```

Resumen claro y directo

☑ ¿Cuántos constructores se ejecutan?

- Todos los de la jerarquía
- Uno por cada clase desde la superclase hasta la subclase

☑ ¿En qué orden?

- **Siempre primero el padre**, luego el hijo

☑ ¿Qué hace **super(...)**?

- Llama explícitamente al constructor de la clase base
- Inicializa el estado heredado

☑ ¿Debo llamar siempre a **super**?

- ✗ No siempre
- ☑ **Sí**, cuando la clase base **no tiene constructor sin parámetros accesible**

3. Respecto a los objetos de subclases en memoria, los atributos privados de la superclase, ¿forman parte de una instancia de la subclase en memoria? En caso afirmativo ¿implica que se puedan usar desde el

código de la subclase? Explícalo con el ejemplo de **Soldado** y alguna de sus subclases.

Sí: **los atributos privados de la superclase sí forman parte de una instancia de la subclase en memoria**, pero **eso NO implica que puedan usarse directamente desde el código de la subclase**. Vamos a separar muy claramente **memoria** de **acceso desde el código**, que es donde suele estar la confusión.

1. ¿Forman parte en memoria? → **Sí, siempre**

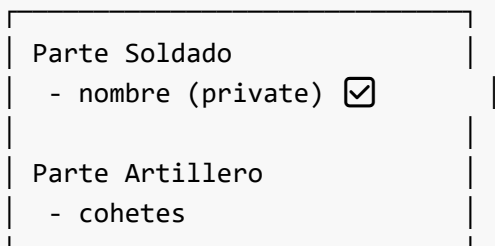
Cuando creas un objeto de una subclase, **el objeto contiene físicamente toda la parte de la superclase**, incluidos:

- atributos **private**
- atributos **protected**
- atributos **public**
- atributos **package-private**

Ejemplo:

```
Artillero a = new Artillero("Lucía", 5);
```

Aunque la referencia sea de tipo **Artillero**, **en memoria existe una única instancia** con esta estructura conceptual:



🔗 **No existen “dos objetos”**, ni uno dentro de otro.
Es **un solo objeto**, con todos los campos de toda la jerarquía.

2. ¿Entonces puedo usar un atributo **private** desde la subclase?

🔒 **NO. El hecho de que exista en memoria no da derecho a accederlo.**

En Java:

private significa **accesible SOLO desde el código de esa clase**, no desde sus hijas.

3. Ejemplo concreto con **Soldado** y **Artillero**

Superclase

```
public class Soldado {  
  
    private String nombre;  
  
    public Soldado(String nombre) {  
        this.nombre = nombre;  
    }  
  
    public void saludar() {  
        System.out.println("Hola, soy " + nombre);  
    }  
}
```

Subclase

```
public class Artillero extends Soldado {  
  
    private int cohetes;  
  
    public Artillero(String nombre, int cohetes) {  
        super(nombre);  
        this.cohetes = cohetes;  
    }  
  
    public void mostrarInfo() {  
        // System.out.println(nombre); ✗ ERROR DE COMPILACIÓN  
        System.out.println("Tengo " + cohetes + " cohetes");  
    }  
}
```

Aunque **nombre** existe dentro del objeto **Artillero**, esta línea:

```
System.out.println(nombre);
```

✗ **no compila**

Motivo:

- **nombre** es **private** en **Soldado**
- **Artillero** **NO tiene permiso** para accederlo directamente

4. ¿Cómo puede entonces la subclase usar ese estado?

Opción correcta: métodos de la superclase

La superclase **sí puede acceder a sus propios campos privados**, y la subclase puede usar esos métodos.

```
public class Soldado {  
  
    private String nombre;  
  
    public Soldado(String nombre) {  
        this.nombre = nombre;  
    }  
  
    public String getNombre() {  
        return nombre;  
    }  
}
```

Ahora:

```
public class Artillero extends Soldado {  
  
    public Artillero(String nombre, int cohetes) {  
        super(nombre);  
    }  
  
    public void presentarse() {  
        System.out.println("Soy el artillero " + getNombre());  
    }  
}
```

- ☒ Funciona
- ☒ Encapsulación respetada
- ☒ Diseño correcto

5. Diferencia clave: herencia $\neq$ visibilidad

Muchos estudiantes piensan:

“Si es heredado, puedo usarlo”

✗ Incorrecto.

Regla importante:

Concepto	Qué significa
Herencia	El atributo forma parte del objeto
private	Solo accesible desde esa clase

Concepto	Qué significa
protected	Accesible desde subclases
public	Accesible desde cualquier sitio

Si **nombre** fuese **protected**:

```
protected String nombre;
```

- ☒ La subclase **sí podría acceder** directamente
- ☒ Pero perderías parte del encapsulamiento (a menudo no recomendable)

6. Conclusión clara y de examen

- ☒ Los atributos privados de la superclase forman parte de la instancia de la subclase en memoria
- ☒ No pueden usarse directamente desde el código de la subclase
- ☒ Se interactúa con ellos a través de métodos públicos o protegidos de la superclase
- ☒ Esto es una decisión de **diseño y encapsulación**, no una limitación técnica de memoria

4. ¿Qué implica en términos de **extensibilidad** de código el hecho de que sean compatibles a nivel de tipos? Ilustra esto añadiendo un nuevo tipo de **Soldado** y demostrando que el código para pedir el saludo a todos los soldados no se modifica.

La **compatibilidad a nivel de tipos** (subtipado) es una de las claves para la **extensibilidad del código** en orientación a objetos.

¿Qué implica para la extensibilidad?

Implica que:

- ☒ Podemos añadir nuevos tipos (subclases) sin modificar el código que trabaja con la superclase.

Esto es fundamental porque:

- Reduce errores
- Evita duplicaciones
- Permite evolucionar el sistema sin tocar código ya probado

En términos de diseño, esto se relaciona con el **principio Abierto/Cerrado**:

El software debe estar abierto a extensión, pero cerrado a modificación.

Recordatorio: compatibilidad de tipos

Si una clase **X** **extiende** a **Soldado**, entonces:

X es-un Soldado

y por tanto:

- Puede guardarse en un **Soldado[]**
- Puede pasarse como parámetro donde se espere un **Soldado**
- Puede usar los métodos definidos en **Soldado**

Código existente (que NO vamos a tocar)

Este código ya existe y funciona:

```
Soldado[] soldados = {  
    new Artillero("Lucía", 5),  
    new Zapador("Miguel", 3)  
};  
  
for (Soldado s : soldados) {  
    s.saludar();  
}
```



Este código no conoce ni le importa el tipo concreto

Solo sabe que todos son **Soldado**.

Añadimos un NUEVO tipo de Soldado

Nuevo subtipo: **Medico**

```
public class Medico extends Soldado {  
  
    private int botiquines;  
  
    public Medico(String nombre, int botiquines) {  
        super(nombre);  
        this.botiquines = botiquines;  
    }  
  
    public int getBotiquines() {  
        return botiquines;  
    }  
}
```

```
public void curar() {  
    System.out.println("Curando a un compañero");  
}  
}
```

Observa:

- **Hereda** `nombre` y `saludar()`
- Añade comportamiento específico (`curar`)
- Cumple la relación **"Medico es-un Soldado"**

Usamos el nuevo tipo SIN CAMBIAR el código de saludo

```
Soldado[] soldados = {  
    new Artillero("Lucía", 5),  
    new Zapador("Miguel", 3),  
    new Medico("Ana", 2)  
};  
  
for (Soldado s : soldados) {  
    s.saludar();  
}
```

☒ El código:

- **No cambia**
- **No se rompe**
- **No necesita saber que existe** `Medico`

Salida posible:

```
Hola, soy el soldado Lucía  
Hola, soy el soldado Miguel  
Hola, soy el soldado Ana
```

¿Por qué esto es extensibilidad real?

Imagina el diseño alternativo (incorrecto):

```
if (s instanceof Artillero) { ... }  
else if (s instanceof Zapador) { ... }
```

Cada nuevo tipo obligaría a:

- Modificar código existente
- Recompilar
- Re-testear

Con herencia y compatibilidad de tipos:

- **Añades clases**
- **No tocas el código común**
- El sistema crece de forma limpia

Resumen claro (respuesta de examen)

- La compatibilidad de tipos permite tratar todas las subclases como la superclase
- Esto hace que el código que opera sobre **Soldado** **no dependa de los tipos concretos**
- Al añadir un nuevo tipo de **Soldado**, el código cliente:
 - ☒ no se modifica
 - ☒ sigue funcionando
- Esto mejora la **extensibilidad**, mantenibilidad y robustez del sistema

5. En Java, cuando trabajo con referencias y herencia. ¿Puedo tener una referencia del supertipo que apunte a objetos reales de un subtipo? ¿Puedo invocar con la referencia del supertipo a métodos públicos del subtipo? ¿En qué consiste el "**upcasting**" y el "**downcasting**"? ¿Qué es el **instanceof**? Pon un ejemplo de recorrido de un array de **Soldado**, comprobando que, si el objeto real es un **Artillero**, solicite el número de cohetes que tiene y los imprima.

1. ¿Puede una referencia del supertipo apuntar a objetos reales de un subtipo?

☒ **Sí, es totalmente válido y es lo normal en Java.**

Ejemplo:

```
Soldado s = new Artillero("Lucía", 5);
```

- **Tipo de la referencia** → **Soldado**
- **Tipo real del objeto en memoria** → **Artillero**

Esto es posible porque:

Artillero es-un Soldado

A esto se le llama **polimorfismo**.

2. ¿Puedo invocar métodos del subtipo usando una referencia del supertipo?

👉 **Depende del tipo de método.**

☒ Sí puedo invocar

- Métodos **definidos en el supertipo** (**Soldado**)
- Métodos **sobrescritos** en el subtipo (polimorfismo dinámico)

```
Soldado s = new Artillero("Lucía", 5);  
s.saludar(); // ☒ permitido
```

Si `saludar()` estuviera sobrescrito en **Artillero**, **se ejecutaría la versión de Artillero**, aunque la referencia sea **Soldado**.

✗ NO puedo invocar directamente

- Métodos que **solo existen en el subtipo**

```
s.getCohetes(); // ✗ ERROR de compilación
```

Motivo:

- El compilador solo mira el **tipo de la referencia** (**Soldado**)
- **Soldado** no tiene `getCohetes()`

3. ¿Qué es el upcasting?

☒ Upcasting = convertir de subtipo a supertipo

```
Artillero a = new Artillero("Lucía", 5);  
Soldado s = a; // upcasting
```

Características:

- ☒ **Seguro**
- ☒ **Implícito** (no hace falta cast)
- ☒ Base del polimorfismo
- **✗** Pierdes acceso a métodos específicos del subtipo

En realidad, **no cambia el objeto**, solo cómo lo miras.

4. ¿Qué es el downcasting?

⚠ Downcasting = convertir de supertipo a subtipo

```
Soldado s = new Artillero("Lucía", 5);  
Artillero a = (Artillero) s; // downcasting
```

Características:

- ✗ **No es seguro por defecto**
- ☒ Requiere *cast explícito*
- ☒ Solo es válido si el objeto real es del subtipo
- ✗ Puede fallar en tiempo de ejecución

Ejemplo peligroso:

```
Soldado s = new Zapador("Miguel", 3);  
Artillero a = (Artillero) s; // ✗ ClassCastException
```

5. ¿Qué es `instanceof`?

`instanceof` sirve para **comprobar el tipo real del objeto en tiempo de ejecución**.

```
if (s instanceof Artillero) {  
    // sabemos que es un Artillero  
}
```

- ☒ Evita errores de `ClassCastException`
- ☒ Se usa antes de hacer *downcasting*
- ✗ Abusarlo suele indicar mal diseño (pero es didáctico y válido)

6. Ejemplo completo: recorrer un array de `Soldado`

Objetivo:

- Todos saludan (polimorfismo)
- Si el soldado real es un `Artillero`, mostramos sus cohetes

Clases (recordatorio mínimo)

```
public class Artillero extends Soldado {  
    private int cohetes;
```

```
public Artillero(String nombre, int cohetes) {
    super(nombre);
    this.cohetes = cohetes;
}

public int getCohetes() {
    return cohetes;
}
}
```

Recorrido del array con `instanceof` y downcasting

```
Soldado[] soldados = {
    new Artillero("Lucía", 5),
    new Zapador("Miguel", 3),
    new Artillero("Ana", 2)
};

for (Soldado s : soldados) {

    // Método común (superclase)
    s.saludar();

    // Comprobamos el tipo real
    if (s instanceof Artillero) {
        Artillero a = (Artillero) s; // downcasting seguro
        System.out.println("Cohetes: " + a.getCohetes());
    }
}
```

Salida posible:

```
Hola, soy el soldado Lucía
Cohetes: 5
Hola, soy el soldado Miguel
Hola, soy el soldado Ana
Cohetes: 2
```

7. Resumen claro para examen

- ☒ Una referencia de supertipo **puede apuntar** a objetos de subtipos
- ☒ Se pueden invocar **solo los métodos visibles en el supertipo**
- ☒ **Upcasting:**
 - Subtipo → Supertipo
 - Implícito y seguro

- ☒ **Downcasting:**
 - Supertipo → Subtipo
 - Explícito y potencialmente peligroso
- ☒ `instanceof` permite comprobar el tipo real antes de hacer downcasting

6. Respecto a la ocultación de información y herencia, ¿qué significa acceso "**protegido**" de métodos y/o atributos? ¿Cómo se implementa en Java? Pon un ejemplo de uso de en la clase `Soldado` para que su nombre sea protegido y pueda usarse en el método de poner bombas del `Zapador`.

¿Qué significa acceso **protegido**?

El acceso **protegido** (`protected`) es un **nivel de visibilidad intermedio** entre `private` y `public`.

Idea clave

Un miembro `protected` **NO es público**, pero **sí es accesible desde las subclases**.

Sirve justamente para este caso:

- ✗ No quiero que cualquiera acceda al atributo
- ☒ Sí quiero que las clases hijas puedan usarlo para implementar su comportamiento

Niveles de acceso en Java (recordatorio rápido)

Modificador	Misma clase	Subclases	Mismo paquete	Cualquiera
<code>private</code>	☒	✗	✗	✗
<code>(default)</code>	☒	✗*	☒	✗
<code>protected</code>	☒	☒	☒	✗
<code>public</code>	☒	☒	☒	☒

* fuera del paquete, solo si es subclase **no** se permite con acceso por defecto.

¿Cómo se implementa en Java?

Simplemente usando la palabra clave `protected`:

```
protected String nombre;
```

¿Qué implica respecto a ocultación de información?

- **protected** sigue siendo encapsulación
- El atributo **no es público**
- Se asume que las subclases:
 - forman parte de la implementación
 - necesitan acceso directo por razones de diseño

🔗 Es una **decisión de diseño consciente**, no un descuido.

Ejemplo con Soldado y Zapador

Clase base **Soldado** con nombre protegido

```
public class Soldado {  
  
    protected String nombre;  
  
    public Soldado(String nombre) {  
        this.nombre = nombre;  
    }  
  
    public void saludar() {  
        System.out.println("Hola, soy el soldado " + nombre);  
    }  
}
```

◊ **nombre:**

- No es **private**
 - No es **public**
 - **Solo accesible por subclases y clases del mismo paquete**
-

Subclase **Zapador** usando el nombre protegido

```
public class Zapador extends Soldado {  
  
    private int minas;  
  
    public Zapador(String nombre, int minas) {  
        super(nombre);  
        this.minas = minas;  
    }  
  
    public void ponerBomba() {  
        System.out.println("El zapador " + nombre + " pone una bomba");  
        minas--;  
    }  
}
```

```
    public int getMinas() {  
        return minas;  
    }  
}
```

☒ Aquí **Zapador** **sí puede usar directamente nombre**, porque:

- Es **protected**
- **Zapador** hereda de **Soldado**

Comparación directa: **private** vs **protected**

Con **private** (NO funciona)

```
private String nombre;
```

```
System.out.println(nombre); // ✗ error de compilación
```

Con **protected** (Sí funciona)

```
protected String nombre;
```

```
System.out.println(nombre); // ☒ correcto
```

¿Es buena práctica usar **protected**?

Depende del diseño:

☒ Tiene sentido cuando:

- El atributo forma parte del **modelo de la clase base**
- Las subclases **necesitan usarlo directamente**
- La jerarquía es estable y bien controlada

☒ No es recomendable cuando:

- Se rompe la encapsulación innecesariamente
- Podría resolverse con métodos (**getNombre()**)

☞ En muchos diseños reales:

- **atributos** → **private**
- **métodos** → **protected**

Pero en ejercicios académicos, **protected** en atributos es muy habitual.

Resumen claro (respuesta de examen)

- El acceso **protegido** (**protected**) permite que un atributo o método:
 - no sea accesible públicamente
 - pero **sí desde subclases**
 - Se implementa en Java con la palabra clave **protected**
 - Un atributo **protected**:
 - forma parte de la instancia heredada
 - puede ser usado directamente por las clases hijas
 - En el ejemplo:
 - **Soldado** define **nombre** como **protected**
 - **Zapador** lo usa en su método **ponerBomba()**
-

7. En los lenguajes orientados a objetos ¿hay una **clase base** para todos los objetos? ¿Ocurre en todos los lenguajes? ¿Qué ocurre en Java?

¿Existe una clase base para todos los objetos en los lenguajes OO?

☞ **Depende del lenguaje orientado a objetos.**

No es una característica obligatoria del paradigma, sino **una decisión de diseño del lenguaje.**

1. En general, en los lenguajes OO

En orientación a objetos **no es obligatorio** que exista una única clase raíz común para todos los objetos.

Lo que **sí** es común al paradigma es que:

- Existan **clases**
- Exista **herencia**
- Se puedan crear **jerarquías de tipos**

Pero:

- ☒ Algunos lenguajes definen una **clase base universal**
 - ☒ Otros **no la hacen explícita** o directamente no la tienen
-

2. Ejemplos en otros lenguajes

☒ Lenguajes con clase base universal

- **Java** → **java.lang.Object**

- **C#** → `System.Object`
- **Python** → `object`
- **Ruby** → `Object`

En estos lenguajes:

- Toda clase hereda (explícita o implícitamente) de esa clase base
- Todos los objetos comparten un conjunto mínimo de métodos

✗ Lenguajes sin una raíz única obligatoria

- **C++**
 - No existe una clase base universal
 - Una clase solo hereda si tú lo indicas
 - Los tipos primitivos ni siquiera son objetos
- **C** (no OO)
 - No hay clases ni herencia

3. ¿Qué ocurre en Java?

☒ En Java **SÍ** existe una clase base común para todos los objetos

Se llama:

```
java.lang.Object
```

Regla fundamental de Java

Toda clase en Java hereda directa o indirectamente de `Object`.

- Aunque no lo escribas
- Aunque no lo sepas
- Siempre ocurre

4. Herencia implícita de `Object`

Si escribes:

```
public class Soldado {  
}
```

Java interpreta realmente:

```
public class Soldado extends Object {  
}
```

Y si luego haces:

```
public class Artillero extends Soldado {  
}
```

La jerarquía real es:

```
Object  
├── Soldado  
    └── Artillero
```

5. ¿Qué aporta **Object**?

Object define métodos **comunes a TODOS los objetos Java**, como por ejemplo:

```
toString()  
equals(Object o)  
hashCode()  
getClass()
```

Esto permite cosas como:

```
Soldado s = new Artillero("Lucía", 5);  
  
System.out.println(s.toString());  
System.out.println(s.getClass());
```

Y funciona **siempre**, porque **Artillero** → **Soldado** → **Object**.

6. Relación con compatibilidad de tipos

Gracias a **Object**:

```
Object o1 = new Soldado("Carlos");  
Object o2 = new Artillero("Lucía", 5);  
Object o3 = new Zapador("Miguel", 3);
```

☑ Cualquier objeto puede almacenarse en una referencia de tipo **Object**.

Esto permite:

- Colecciones genéricas
- Librerías reutilizables
- APIs comunes
- Polimorfismo a nivel máximo

7. ¿Y los tipos primitivos en Java?

⚠ Importante matiz:

- **int, double, boolean**, etc.
- **NO heredan de Object**
- **NO son objetos**

Pero Java ofrece **clases envoltorio** (*wrapper classes*):

Primitivo	Envoltorio
int	Integer
double	Double
boolean	Boolean

Estas **sí heredan de Object**.

8. Resumen claro (respuesta tipo examen)

- ? ¿Hay una clase base para todos los objetos en POO?
 - → **No es obligatorio** en todos los lenguajes
 - ☑ Algunos lenguajes la definen explícitamente
 - ☑ **En Java sí existe: `java.lang.Object`**
 - ☑ Toda clase Java hereda directa o indirectamente de **Object**
 - ☑ Esto proporciona métodos comunes y una raíz del sistema de tipos
 - ✗ Los tipos primitivos no heredan de **Object**
-

8. ¿Qué es la "**herencia múltiple**"? ¿Existe en Java herencia múltiple?

¿Qué es la **herencia múltiple**?

La **herencia múltiple** es una característica de algunos lenguajes orientados a objetos que permite que **una clase herede directamente de más de una clase base**.

Dicho de otra forma:

Una clase puede ser **hija de varias superclases al mismo tiempo**.

Ejemplo conceptual (no Java)

Clase C hereda de A y de B

```
class C extends A, B {    // ← herencia múltiple
}
```

Esto significa que:

- C heredaría atributos y métodos de A
- **y también** atributos y métodos de B

¿Qué ventajas tiene la herencia múltiple?

☒ Permite:

- Reutilizar código de varias jerarquías
- Modelar objetos que "son varias cosas a la vez"

¿Qué problemas plantea?

El problema más famoso es el **problema del diamante** (*diamond problem*).

Ejemplo del problema

```
class A {
    void saludar() { }
}

class B extends A {
    void saludar() { }
}

class C extends A {
    void saludar() { }
}

class D extends B, C {    // ← herencia múltiple
}
```

? Pregunta inevitable:

- Si llamamos a `saludar()` en D...
 - ¿se ejecuta el de B?
 - ¿el de C?
 - ¿el de A?

☞ Ambigüedad → complejidad → errores difíciles de entender.

¿Existe herencia múltiple en Java?

✗ NO existe herencia múltiple de clases en Java

En Java:

```
class Artillero extends Soldado, Unidad { // ✗ NO permitido
}
```

Esto **no compila**.

Regla clave de Java

Una clase Java solo puede extender de UNA única clase.

Esto es una decisión de diseño **consciente**, para:

- Simplificar el lenguaje
 - Evitar ambigüedades
 - Facilitar el razonamiento y el mantenimiento del código
-

¿Entonces Java no permite herencia múltiple en absoluto?

☞ **Sí la permite, pero SOLO a través de interfaces.**

Herencia múltiple **con interfaces** (la forma Java)

Una clase puede:

- Extender **una sola clase**
- Implementar **múltiples interfaces**

```
class Zapador extends Soldado
    implements Minador, Ingeniero {
}
```

¿Por qué con interfaces sí?

Porque las interfaces:

- No tienen estado (atributos de instancia)
- Definen **qué se puede hacer**, no **cómo se almacena**
- Evitan conflictos de atributos

¿Y los métodos en interfaces?

Desde Java 8:

- Las interfaces pueden tener **métodos default**
- Aun así, Java obliga a resolver conflictos explícitamente

Ejemplo:

```
interface A {
    default void saludar() {
        System.out.println("Hola desde A");
    }
}

interface B {
    default void saludar() {
        System.out.println("Hola desde B");
    }
}

class C implements A, B {

    @Override
    public void saludar() {
        A.super.saludar(); // resolución explícita
    }
}
```

- ☒ No hay ambigüedad implícita
- ☒ El programador decide

Comparación clara

Característica	Clases	Interfaces
Herencia múltiple	✗ No	☒ Sí
Atributos de instancia	☒ Sí	✗ No
Constructores	☒ Sí	✗ No
Resolución de conflictos	Problemática	Explícita

Relación con el diseño orientado a objetos

Java promueve:

“Herencia simple + composición + interfaces”

En vez de:

- Heredar de muchas clases
- Crear jerarquías complejas y frágiles

Resumen claro (respuesta de examen)

- ☒ **La herencia múltiple** permite que una clase herede de varias clases base
 - ☒ **Java NO permite herencia múltiple de clases**
 - ☒ Java **sí permite herencia múltiple de interfaces**
 - ☒ Se evita el problema del diamante y las ambigüedades
 - ☒ Es una decisión de diseño para favorecer simplicidad y robustez
-

9. Las excepciones en los lenguajes orientados a objetos son objetos. Por tanto, se pueden crear excepciones personalizadas. Pon un ejemplo en Java de una excepción personalizada (**UsuarioNoEncontradoException**), que sea *no controlada* y que además este compuesto con un **Usuario**, para saber qué **Usuario** dio el problema. Permite además que se pueda incluir la causa, es decir, sobrecarga el constructor para tener una versión que permita añadir la causa subyacente.

1. Excepciones como objetos en Java

En Java:

- **Todas las excepciones son objetos**
- Todas heredan directa o indirectamente de **Throwable**
- Las excepciones **no controladas** heredan de **RuntimeException**

☞ Una excepción *no controlada*:

- **No obliga** a usar **try/catch**
 - Se usa para errores de programación o estados inesperados
-

2. Requisitos de la excepción

Queremos una excepción:

- ☒ Personalizada: **UsuarioNoEncontradoException**
- ☒ **No controlada** → extiende **RuntimeException**
- ☒ **Compuesta** con un **Usuario** (referencia al objeto problemático)
- ☒ Con **sobrecarga de constructores**:

- Uno sin causa
 - Otro con causa (`Throwable cause`)
-

3. Clase `Usuario` (modelo simple)

```
public class Usuario {  
  
    private String nombre;  
  
    public Usuario(String nombre) {  
        this.nombre = nombre;  
    }  
  
    public String getNombre() {  
        return nombre;  
    }  
}
```

4. Excepción personalizada no controlada

`UsuarioNoEncontradoException`

```
public class UsuarioNoEncontradoException extends RuntimeException {  
  
    private final Usuario usuario;  
  
    // Constructor sin causa  
    public UsuarioNoEncontradoException(Usuario usuario) {  
        super("Usuario no encontrado: " + usuario.getNombre());  
        this.usuario = usuario;  
    }  
  
    // Constructor con causa subyacente  
    public UsuarioNoEncontradoException(Usuario usuario, Throwable cause) {  
        super("Usuario no encontrado: " + usuario.getNombre(), cause);  
        this.usuario = usuario;  
    }  
  
    public Usuario getUsuario() {  
        return usuario;  
    }  
}
```

5. Aspectos OO importantes del diseño

☒ Es **no controlada**

```
extends RuntimeException
```

No obliga a capturarla → decisión consciente de diseño.

☒ Está **compuesta** con un **Usuario**

```
private final Usuario usuario;
```

La excepción **contiene información de dominio**, no solo un mensaje.

☒ Permite encadenamiento de causas (*exception chaining*)

```
super(mensaje, cause);
```

Esto permite:

- Saber **qué usuario falló**
 - Saber **qué excepción provocó el fallo original**
-

6. Ejemplo de uso

```
public class ServicioUsuarios {  
  
    public Usuario buscarUsuario(String nombre) {  
        Usuario usuario = new Usuario(nombre);  
  
        boolean existe = false; // simulación  
  
        if (!existe) {  
            throw new UsuarioNoEncontradoException(usuario);  
        }  
  
        return usuario;  
    }  
}
```

Con causa subyacente

```
try {  
    // código que falla (por ejemplo acceso a BD)  
    throw new IllegalStateException("Error en base de datos");  
} catch (IllegalStateException e) {  
    throw new UsuarioNoEncontradoException(new Usuario("Carlos"), e);  
}
```

7. Ventajas del enfoque

- ☒ Excepciones **ricas en información**
 - ☒ Mejor depuración y trazabilidad
 - ☒ Diseño orientado a objetos real (no solo `String`)
 - ☒ Compatible con logging y análisis posterior
-

8. Resumen (respuesta tipo examen)

- En Java, las excepciones son objetos y pueden personalizarse
 - Una excepción **no controlada** hereda de `RuntimeException`
 - Las excepciones pueden:
 - contener objetos del dominio
 - encadenar causas (`Throwable cause`)
 - `UsuarioNoEncontradoException`:
 - encapsula un `Usuario`
 - ofrece constructores con y sin causa
 - permite identificar tanto **el objeto implicado** como **el origen del error**
-

10. Herencia vs. Composición. Se dice que no se debe emplear herencia simplemente por reutilizar código, es decir, que si quiero reutilizar código simplemente, no debo pensar en herencia como primera opción ¿por qué?

1. Herencia no es solo reutilización de código

La herencia **no significa simplemente “copiar código”**.

La herencia significa afirmar una relación **conceptual fuerte**:

A es-un B (*is-a*)

Si usas herencia, estás diciendo que:

- El subtipo puede sustituir al supertipo
- Comparte su significado
- Cumple su contrato

Si **solo quieres reutilizar implementación**, esa relación **puede no existir**.

➡ Usar herencia solo por reutilización **fuerza una relación semántica falsa.**

2. Problema principal: acoplamiento fuerte

Cuando una clase hereda de otra:

- Queda **fuertemente acoplada** a la implementación del padre
- Cambios en la superclase pueden romper las subclases
- Se introduce el problema de la **clase base frágil**

Ejemplo conceptual:

```
class Soldado {  
    protected int energia;  
  
    public void descansar() {  
        energia += 10;  
    }  
}
```

Si más tarde cambias `descansar()`:

- Todas las subclases cambian de comportamiento
- Aunque no querías tocar su lógica

➡ **La herencia propaga cambios** de forma implícita y peligrosa.

3. La herencia expone más de lo necesario

Con herencia, la subclase:

- Ve métodos `protected`
- Está tentada de depender de detalles internos
- Se vuelve frágil ante refactorizaciones

Eso rompe uno de los principios clave de OO:

☑ **Encapsulación**

4. Composición: reutilizas comportamiento sin mentir al modelo

Composición = **tiene-un** (*has-a*)

En vez de heredar:

```
class Zapador extends Soldado { ... }
```

Compones:

```
class Zapador {  
    private Soldado soldado;  
}
```

Ventajas:

- ☒ Reutilizas código
- ☒ No fuerzas una jerarquía semántica
- ☒ Menor acoplamiento
- ☒ Más flexibilidad

5. Ejemplo claro: reutilización mal aplicada con herencia

✗ Mal uso de herencia

```
class Motor {  
    void arrancar() { ... }  
}  
  
class Coche extends Motor { // ✗  
}
```

Un coche **NO es un motor**, aunque lo use.

Esto rompe el modelo conceptual.

☒ Uso correcto: composición

```
class Motor {  
    void arrancar() { ... }  
}  
  
class Coche {  
    private Motor motor;  
  
    void arrancar() {  
        motor.arrancar();  
    }  
}
```

Aquí:

- El coche **tiene un motor**

- No depende de su implementación interna
- Puede cambiar de motor sin romper nada

6. Herencia reduce flexibilidad futura

Con herencia:

- La relación está fijada **en tiempo de compilación**
- Cambiar la jerarquía suele ser caro

Con composición:

- Puedes cambiar componentes dinámicamente
- Puedes combinar comportamientos

Ejemplo:

```
Soldado soldado = new Soldado();  
soldado.setArma(new Lanzacohetes());
```

Mucho más flexible que crear subclases para cada combinación.

7. ¿Cuándo Sí usar herencia?

☒ La herencia es correcta cuando:

- Hay una relación clara **“es-un”**
- El subtipo cumple el **principio de sustitución**
- Quieres **polimorfismo**, no solo reutilización
- La jerarquía es estable y conceptual

Ejemplo correcto:

```
Artillero extends Soldado  
Zapador extends Soldado
```

8. Idea central (muy típica de examen)

Herencia expresa una relación de tipos, composición expresa una relación de uso.

9. Resumen claro y directo

- **✗** No se debe usar herencia solo para reutilizar código porque:
 - Fuerza relaciones conceptuales falsas

- Introduce acoplamiento fuerte
 - Reduce encapsulación
 - Hace el código más frágil ante cambios
 - ☒ La composición:
 - Reutiliza comportamiento sin comprometer el modelo
 - Es más flexible y mantenible
 - ☒ La herencia debe usarse cuando existe una relación real **"es-un"**, no solo por conveniencia
-

11. Herencia vs. Composición. Se dice que se debe *"favorecer la composición frente a la herencia"*, ¿por qué?

1. Qué significa exactamente la frase

Favorecer la composición frente a la herencia significa:

☞ *Antes de crear una subclase, piensa si lo que necesitas puede resolverse componiendo objetos.*

O dicho en términos OO clásicos:

- **Herencia** → relación *"es-un"*
 - **Composición** → relación *"tiene-un / usa-un"*
-

2. Porque la herencia crea un acoplamiento fuerte

Cuando una clase **hereda** de otra:

- Depende **directamente** de su implementación
- Cambios en la clase base pueden afectar a todas las subclases
- Aparece el problema de la **clase base frágil**

Ejemplo conceptual:

```
class Soldado {  
    protected int energia;  
  
    public void descansar() {  
        energia += 10;  
    }  
}
```

Si mañana cambias la lógica de `descansar()`:

- Todas las subclases se ven afectadas
- Aunque conceptualmente no querías cambiar su comportamiento

☞ La herencia **propaga cambios de forma implícita**.

3. Porque la herencia rompe fácilmente la encapsulación

La herencia:

- Expone detalles internos mediante `protected`
- Invita a que la subclase dependa de cómo está implementado el padre
- Hace difícil refactorizar sin romper código

Esto va contra un principio básico de OO:

☒ **Encapsulación: ocultar los detalles internos**

Con **composición**, el acceso es explícito y controlado.

4. Porque la herencia fija el diseño en tiempo de compilación

La relación de herencia es:

- **Estática**
- **Difícil de cambiar**
- Costosa si el diseño evoluciona

En cambio, la composición:

- Permite cambiar componentes
- Permite combinar comportamientos dinámicamente

Ejemplo:

```
class Soldado {  
    private Arma arma;  
  
    public void setArma(Arma arma) {  
        this.arma = arma;  
    }  
}
```

Aquí puedes:

- Cambiar el arma en ejecución
- Añadir nuevas armas sin tocar `Soldado`

Con herencia necesitarías:

```
SoldadoConPistola  
SoldadoConRifle  
SoldadoConRifleYEscudo
```

🔗 Explosión de clases.

5. Porque la herencia suele forzar relaciones conceptuales falsas

Usar herencia solo por reutilizar código lleva a errores de modelado:

✗ Ejemplo incorrecto:

```
class Motor { }  
class Coche extends Motor { } // un coche NO es un motor
```

☑ Ejemplo correcto:

```
class Coche {  
    private Motor motor;  
}
```

La composición **no miente sobre el dominio**, la herencia sí puede hacerlo.

6. Porque la composición es más flexible y escalable

Con composición:

- Puedes añadir nuevas clases sin tocar las existentes
- Sigues mejor el principio **abierto/cerrado**
- El sistema crece sin rigidez

Este enfoque es la base de:

- Patrones de diseño (Strategy, Decorator, Composite...)
 - Arquitecturas modernas
 - Sistemas extensibles
-

7. Entonces... ¿la herencia es mala?

✗ **NO.**

La herencia es **correcta y potente** cuando:

- ☑ Existe una relación real **“es-un”**
- ☑ Se quiere polimorfismo por tipos
- ☑ La jerarquía es estable
- ☑ El subtipo cumple el **principio de sustitución**

Ejemplo correcto:

```
Artillero extends Soldado
Zapador extends Soldado
```

Pero **no debe usarse como herramienta de reutilización automática.**

8. Idea central que suelen buscar en exámenes

La herencia modela tipos. La composición modela comportamiento reutilizable.

Y por eso:

Se favorece la composición porque da más flexibilidad, menos acoplamiento y mejor encapsulación.

9. Resumen final (respuesta perfecta de examen)

- Se favorece la composición frente a la herencia porque:
 - Reduce el acoplamiento entre clases
 - Preserva mejor la encapsulación
 - Evita jerarquías rígidas y frágiles
 - Permite mayor flexibilidad y evolución del sistema
 - La herencia debe usarse solo cuando existe una relación clara **"es-un"**
 - La composición permite reutilizar código **sin forzar relaciones conceptuales falsas**
-
-

12. Herencia vs. Composición. Se dice que la *"herencia rompe la encapsulación"*, ¿a qué se refiere esto?

Cuando se afirma que **"la herencia rompe la encapsulación"** no se quiere decir que *siempre* sea mala, sino que **la herencia introduce una grieta en uno de los principios fundamentales de la orientación a objetos: ocultar los detalles internos**. Vamos a explicarlo con precisión y ejemplos.

1. ¿Qué es encapsulación?

La **encapsulación** consiste en:

Ocultar los detalles internos de una clase y exponer solo lo necesario mediante una interfaz estable.

En Java esto se logra con:

- Atributos **private**
- Métodos públicos como única forma de interacción
- Separación clara entre **qué hace una clase** y **cómo lo hace**

Ejemplo ideal:

```
class Soldado {  
    private int energia;  
  
    public void descansar() {  
        energia += 10;  
    }  
}
```

El usuario de **Soldado**:

- No sabe cómo se almacena **energia**
- Solo sabe que puede llamar a **descansar()**

☒ Encapsulación intacta.

2. ¿Dónde entra la herencia?

Cuando una clase **hereda**, pasa algo distinto:

```
class Artillero extends Soldado {  
    // acceso a métodos y atributos del padre  
}
```

La subclase:

- **No es un simple usuario**
- Se convierte en un **colaborador interno**
- Necesita conocer más sobre la implementación de la clase base

🔗 Aquí aparece el problema.

3. ¿Por qué se dice que la herencia “rompe” la encapsulación?

Razón principal:

La herencia obliga a exponer detalles internos de la clase base a sus subclases.

Esto ocurre especialmente a través de:

- Atributos **protected**
- Métodos **protected**
- Dependencia del orden y efecto de llamadas internas

4. Ejemplo claro con **protected**

```
class Soldado {  
    protected int energia;  
  
    protected void gastarEnergia(int cantidad) {  
        energia -= cantidad;  
    }  
}
```

```
class Zapador extends Soldado {  
  
    public void ponerBomba() {  
        gastarEnergia(30);  
        energia -= 5; // acceso directo al estado interno  
    }  
}
```

Aquí:

- **Zapador conoce y manipula el estado interno** de **Soldado**
- La implementación ya **no está oculta**
- La encapsulación queda debilitada

5. Consecuencia: clase base frágil

Una vez una subclase depende de detalles internos:

```
protected int energia;
```

La clase base **ya no puede cambiar libremente**:

- Cambiar **energia** por otro modelo → rompe subclases
- Alterar invariantes internas → comportamiento inesperado
- Refactorizar se vuelve peligroso

Esto se conoce como el **problema de la clase base frágil** (*fragile base class problem*).

☞ La clase base deja de ser realmente encapsulada.

6. Herencia vs. uso normal de una clase

Cliente normal (composición)

```
class Mision {  
    private Soldado soldado;
```

```
public void ejecutar() {  
    soldado.descansar();  
}
```

- Solo ve métodos públicos
- No depende de implementación
- Encapsulación respetada

Subclase (herencia)

```
class Artillero extends Soldado {  
    public void atacar() {  
        energia -= 20; // dependencia interna  
    }  
}
```

- Ve más de lo debido
- Puede romper invariantes
- Depende de decisiones internas

✗ Encapsulación debilitada

7. ¿La herencia rompe *siempre* la encapsulación?

✗ **No siempre**, pero:

- **La debilita**
- Introduce una **excepción controlada** al encapsulamiento
- Debe usarse **solo cuando la relación “es-un” es muy sólida**

Por eso se dice:

La herencia es una relación fuerte y privilegiada.

8. ¿Por qué la composición no rompe la encapsulación?

Con composición:

```
class Zapador {  
    private Soldado soldado;  
  
    public void ponerBomba() {  
        soldado.descansar();  
    }  
}
```



```
}  
}
```

- Solo se usa la **interfaz pública**
- No se accede al estado interno
- La implementación puede cambiar libremente

☒ Encapsulación intacta.

9. Idea clave que se quiere transmitir con la frase

Cuando se dice “**la herencia rompe la encapsulación**” se quiere decir:

La herencia obliga a revelar detalles internos al código heredado, creando acoplamiento y reduciendo la libertad de evolución de la clase base.

No es una prohibición, sino una **advertencia de diseño**.

10. Resumen claro (respuesta de examen)

- La encapsulación busca ocultar la implementación interna de una clase
- La herencia permite a las subclasses acceder a miembros **protected** y depender del comportamiento interno
- Esto crea:
 - Acoplamiento fuerte
 - Dependencia de detalles de implementación
 - Fragilidad ante cambios
- Por ello se dice que **la herencia debilita o rompe la encapsulación**
- La composición evita este problema al interactuar solo mediante interfaces públicas

13. Pongamos un ejemplo de dos alternativas para lo mismo. Tenemos un **Estudiante** y un **Trabajador**, ambos tienen datos en común: el DNI y el nombre. Modelemos esto de dos formas: uno por herencia, con una superclase **Persona**, y otro con composición, con una clase **DatosPersonales**. Se debe recibir una instancia de **DatosPersonales** en el constructor de la clase **Estudiante** y **Trabajador**.

Planteamiento del problema

Tenemos dos clases:

- **Estudiante**
- **Trabajador**

Ambas comparten datos:

- **dni**

- nombre

Vamos a resolverlo de dos maneras:

1. ☒ **Herencia** → superclase **Persona**
2. ☒ **Composición** → clase **DatosPersonales** usada por ambas

☒ Opción 1: Modelado mediante **herencia**

Aquí afirmamos explícitamente:

Un Estudiante es una Persona
Un Trabajador es una Persona

Es una relación clara “es-un”.

Superclase **Persona**

```
public class Persona {  
  
    private String dni;  
    private String nombre;  
  
    public Persona(String dni, String nombre) {  
        this.dni = dni;  
        this.nombre = nombre;  
    }  
  
    public String getDni() {  
        return dni;  
    }  
  
    public String getNombre() {  
        return nombre;  
    }  
}
```

Subclase **Estudiante**

```
public class Estudiante extends Persona {  
  
    private String carrera;  
  
    public Estudiante(String dni, String nombre, String carrera) {  
        super(dni, nombre);  
        this.carrera = carrera;  
    }  
}
```

```
    }

    public String getCarrera() {
        return carrera;
    }
}
```

Subclase **Trabajador**

```
public class Trabajador extends Persona {

    private String puesto;

    public Trabajador(String dni, String nombre, String puesto) {
        super(dni, nombre);
        this.puesto = puesto;
    }

    public String getPuesto() {
        return puesto;
    }
}
```

☒ Características de esta solución

- Reutilización mediante herencia
- Polimorfismo posible (`Persona p = new Estudiante(...)`)
- Relación fuerte y estática
- Afirmación semántica clara (*es-un*)

☒ Opción 2: Modelado mediante **composición**

Aquí **no afirmamos** que Estudiante o Trabajador *sean* datos personales.

Decimos:

```
Un Estudiante tiene datos personales
Un Trabajador tiene datos personales
```

Relación “*tiene-un*”.

Clase **DatosPersonales**

```
public class DatosPersonales {  
  
    private String dni;  
    private String nombre;  
  
    public DatosPersonales(String dni, String nombre) {  
        this.dni = dni;  
        this.nombre = nombre;  
    }  
  
    public String getDni() {  
        return dni;  
    }  
  
    public String getNombre() {  
        return nombre;  
    }  
}
```

Clase **Estudiante** (composición)

```
public class Estudiante {  
  
    private DatosPersonales datos;  
    private String carrera;  
  
    public Estudiante(DatosPersonales datos, String carrera) {  
        this.datos = datos;  
        this.carrera = carrera;  
    }  
  
    public String getCarrera() {  
        return carrera;  
    }  
  
    public DatosPersonales getDatos() {  
        return datos;  
    }  
}
```

Clase **Trabajador** (composición)

```
public class Trabajador {  
  
    private DatosPersonales datos;  
    private String puesto;
```

```
public Trabajador(DatosPersonales datos, String puesto) {
    this.datos = datos;
    this.puesto = puesto;
}

public String getPuesto() {
    return puesto;
}

public DatosPersonales getDatos() {
    return datos;
}
}
```

Ejemplo de uso

```
DatosPersonales dp = new DatosPersonales("12345678A", "Laura");

Estudiante e = new Estudiante(dp, "Ingeniería Informática");
Trabajador t = new Trabajador(dp, "Programadora");
```

☒ Características de esta solución

- No hay herencia
- Menor acoplamiento
- Mayor flexibilidad
- Mejor encapsulación
- Reutilización *sin forzar* una jerarquía conceptual

Comparación directa

Aspecto	Herencia	Composición
Relación	es-un	tiene-un
Acoplamiento	Alto	Bajo
Flexibilidad	Menor	Mayor
Encapsulación	Más débil	Más fuerte
Polimorfismo	Sí	No (directo)
Evolución del diseño	Más frágil	Más estable

☑ Conclusión conceptual (muy de examen)

- La **herencia** es adecuada cuando existe una relación clara de tipo (*"es-un"*).
 - La **composición** es preferible cuando solo se quiere:
 - reutilizar datos o comportamiento
 - evitar jerarquías rígidas
 - mantener encapsulación y flexibilidad
 - Ambos modelos son correctos; la elección depende del **modelo del dominio**, no solo del código.
-
-